



Case Study based on Linked List:-

Air Traffic Control (ATC)

Team members :-

- 1) Danish Sayyad - 124B1B100
- 2) Khushi Dudhalkar - 124B1B102
- 3) Diksha Kate - 124B1B108
- 4) Piyush Patel – 124B1B112

Course:- Data Structure

Course Teacher:- Prof. Ashwini Matange

Problem Statement:-

As global air traffic increases, current Air Traffic Control (ATC) systems struggle to manage real-time data, flight coordination, and complex airspace efficiently. There is a critical need for advanced, automated software solutions that enhance safety, reduce controller workload, and improve decision-making through real-time data integration, predictive analytics, and intelligent flight tracking.

Data Structure concepts used in this case study:

Linked Lists

- Both Airport and Flight structures use **singly linked lists**.
- Airports are linked through the next pointer in Airport.
- Each Airport maintains a list of Flights linked by the next pointer in Flight.

Dynamic Memory Allocation

- The program dynamically allocates memory using new for both airports and flights.
- Memory is explicitly deallocated using delete in the destructor and removal functions.

Structs

- Flight and Airport are defined using struct, a fundamental data structure for grouping data members.

Encapsulation (OOP)

- The ATC class encapsulates all operations and maintains the integrity of the airport-flight system.

Core Features:

Airport Management

- Add new airports.
- Remove existing airports along with their flights.
- Display all airports and their flights.

Flight Management

- Add a flight to a specific airport (arrival).
- Delete a flight (departure).
- Update flight destination.
- Track number of flights per airport.

Flight Limit Control

- Each airport is limited to a **maximum of 10 flights** (MAX_FLIGHTS constant).

Persistence

- Save the entire system state to a file.
- Load state back from the file (with complete structure restoration).

Technical Details:

Language: C++

Data Structures: Nested Linked List

Flight Limit: Max 10 flights per airport (MAX_FLIGHTS)

Memory Management: Manual using new and delete

File I/O: ifstream and ofstream used to save/load data from a text file

Interface: Console-based menu-driven system

Persistence: Supports saving/loading complete airport and flight data

Key Classes:

Flight(struct):

Stores flight number, destination, status, and pointer to the next flight.

Airport(struct):

Stores airport code, head of flight list, flight count, and pointer to the next airport.

ATC(class):

Manages all operations including adding/removing airports, managing flights, displaying data, and file operations. Encapsulates core logic.

Time and Space Complexity:-

Let **A** be the number of airports.

Let **F** be the number of flights per airport (max 10 per implementation).

Function	Time Complexity	Space Complexity
<code>addAirport()</code>	$O(A)$	$O(1)$
<code>removeAirport()</code>	$O(A + F)$	$O(1)$
<code>addFlight()</code>	$O(A + F)$	$O(1)$
<code>deleteFlight()</code>	$O(A + F)$	$O(1)$
<code>updateFlight()</code>	$O(A + F)$	$O(1)$
<code>display()</code>	$O(A * F)$	$O(1)$
<code>saveToFile()</code>	$O(A * F)$	$O(1)$
<code>loadFromFile()</code>	$O(A * F)$	$O(A * F)$
<code>flightExists()</code>	$O(F)$	$O(1)$
<code>findAirport()</code>	$O(A)$	$O(1)$
Destructor <code>~ATC()</code>	$O(A * F)$	$O(1)$

Software Requirements:

Operating System:Linux

Compiler: g++ (GCC)

C++ Version: C++11 or higher

Terminal/Console: For running the program

./.

Hardware Requirements:

Processor: Any modern CPU

RAM: 2 GB minimum

Storage: Minimal – used only for the compiler

Display: Basic terminal or console

Source Code:-

```
// Air control system with flight limit per airport
```

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <iomanip>
```

```
using namespace std;
```

```
// Flight node structure
```

```
struct Flight {
```

```
    string flightNo;
```

```
    string destination;
```

```
    string status;
```

```
    Flight *next;
```

```
};
```

```
// Airport node structure
```

```
struct Airport {
```

```
    string airportCode;
```

```
    Flight *flightHead;
```

```
    int flightCount;
```

```
    Airport *next;
```

```
};
```

```
// Air Traffic Control system
```

```
class ATC {
```

```
private:
```

```
    Airport *airportHead;
```

```
    const int MAX_FLIGHTS = 10;
```

```
// Helper function to check if flight exists in given airport
```

```
bool flightExists(Airport* airport, const string& flightNum) {
```

```
    Flight* current = airport->flightHead;
```

```
    while (current) {
```

```
        if (current->flightNo == flightNum)
```

```
            return true;
```

```
            current = current->next;
```

```
    }
```

```
    return false;
```

```
}
```

```
void addFlight_FromLoad(string code, string flightnum, string dest, string status) {
```

```
    Airport *airport = findAirport(code);
```

```
    if (!airport) return;
```

```
    if (flightExists(airport, flightnum)) {
```

```
        cout << "Duplicate flight " << flightnum << " in " << code << " skipped during load.\n";
```

```
        return;
```

```
    }
```

```
    Flight *f_new = new Flight{flightnum, dest, status, NULL};
```

```
    if (!airport->flightHead) {
```

```
        airport->flightHead = f_new;
```

```

    } else {
        Flight *temp = airport->flightHead;
        while (temp->next)
            temp = temp->next;
        temp->next = f_new;
    }
    airport->flightCount++;
}

```

public:

```

ATC() {
    airportHead = NULL;
}

```

```

void addAirport(string code) {
    if (findAirport(code)) {
        cout << "Airport with code " << code << " already exists.\n";
        return;
    }
}

```

```

Airport *newAirport = new Airport{code, NULL, 0, NULL};
if (!airportHead) {
    airportHead = newAirport;
} else {
    Airport *temp = airportHead;
    while (temp->next)
        temp = temp->next;
    temp->next = newAirport;
}
cout << "Airport " << code << " added.\n";

```

```
}
```

```
void removeAirport(string code) {
```

```
    Airport *cu = airportHead, *pr = NULL;
```

```
    while (cu) {
```

```
        if (cu->airportCode == code) {
```

```
            if (pr)
```

```
                pr->next = cu->next;
```

```
            else
```

```
                airportHead = cu->next;
```

```
        while (cu->flightHead) {
```

```
            Flight *fTemp = cu->flightHead;
```

```
            cu->flightHead = fTemp->next;
```

```
            delete fTemp;
```

```
        }
```

```
        delete cu;
```

```
        cout << "Airport " << code << " removed.\n";
```

```
        return;
```

```
    }
```

```
    pr = cu;
```

```
    cu = cu->next;
```

```
}
```

```
    cout << "Airport not found!\n";
```

```
}
```

```
void addFlight(string code, string flightNum, string dest) {
```

```
    Airport *airport = findAirport(code);
```

```
    if (!airport) {
```

```
        cout << "Airport not found.\n";
```

```

        return;
    }

    if (airport->flightCount >= MAX_FLIGHTS) {
        cout << "Cannot add more flights to " << code << ". Maximum limit of " <<
MAX_FLIGHTS << " reached.\n";
        return;
    }

    if (flightExists(airport, flightNum)) {
        cout << "Flight " << flightNum << " already exists in " << code << ".\n";
        return;
    }

    Flight *newFlight = new Flight{flightNum, dest, "On Time", NULL};
    if (!airport->flightHead) {
        airport->flightHead = newFlight;
    } else {
        Flight *temp = airport->flightHead;
        while (temp->next)
            temp = temp->next;
        temp->next = newFlight;
    }

    airport->flightCount++;
    cout << "Flight " << flightNum << " to " << dest << " added to " << code << ".\n";
}

void deleteFlight(string code, string flightNum) {
    Airport *airport = findAirport(code);
    if (!airport) {

```

```

        cout << "Airport not found!\n";
        return;
    }

    Flight *cu = airport->flightHead;
    Flight *pr = NULL;

    while (cu) {
        if (cu->flightNo == flightNum) {
            if (pr)
                pr->next = cu->next;
            else
                airport->flightHead = cu->next;
            delete cu;
            airport->flightCount--;
            cout << "Flight " << flightNum << " deleted from " << code << ".\n";
            return;
        }
        pr = cu;
        cu = cu->next;
    }
    cout << "Flight not found!\n";
}

void updateFlight(string code, string flightNum, string newDest) {
    Airport *airport = findAirport(code);
    if (!airport) {
        cout << "Airport not found!\n";
        return;
    }
}

```

```

Flight *flight = airport->flightHead;
while (flight) {
    if (flight->flightNo == flightNum) {
        flight->destination = newDest;
        cout << "Flight " << flightNum << " updated.\n";
        return;
    }
    flight = flight->next;
}
cout << "Flight not found!\n";
}

void display() {
    Airport *aTemp = airportHead;
    while (aTemp) {
        cout << "\nAirport: " << aTemp->airportCode
            << " | Total Flights: " << aTemp->flightCount << "\n";

        Flight *fTemp = aTemp->flightHead;
        if (!fTemp) {
            cout << " No flights scheduled.\n";
        } else {
            while (fTemp) {
                cout << " Flight: " << fTemp->flightNo
                    << " -> " << fTemp->destination
                    << " | Status: " << fTemp->status << "\n";
                fTemp = fTemp->next;
            }
        }
        aTemp = aTemp->next;
    }
}

```

```
}  
}
```

```
void saveToFile(const string& filename) {  
    ofstream outfile(filename);  
    if (!outfile.is_open()) {  
        cout << "Error opening file for saving\n";  
        return;  
    }  
}
```

```
Airport *currentAirport = airportHead;
```

```
while (currentAirport) {
```

```
    outfile  
    "===== \n"; <<
```

```
    outfile << "Airport Code: " << currentAirport->airportCode << "\n";
```

```
    outfile << "Total Flights: " << currentAirport->flightCount << "\n";
```

```
    outfile << "----- \n";
```

```
    outfile << left << setw(15) << "Flight No"
```

```
        << setw(25) << "Destination"
```

```
        << setw(15) << "Status" << "\n";
```

```
    outfile << "----- \n";
```

```
Flight *currentFlight = currentAirport->flightHead;
```

```
if (!currentFlight) {
```

```
    outfile << "No flights scheduled.\n";
```

```
} else {
```

```
    while (currentFlight) {
```

```
        outfile << left << setw(15) << currentFlight->flightNo
```

```
            << setw(25) << currentFlight->destination
```

```
            << setw(15) << currentFlight->status << "\n";
```

```
        currentFlight = currentFlight->next;
```

```

    }
}

    outfile
"=====\\n\\n";
    currentAirport = currentAirport->next;
}

outfile.close();
cout << "Data successfully saved to: " << filename << endl;
}

void loadFromFile(const string& filename) {
    ifstream infile(filename);
    if (!infile.is_open()) {
        cout << "No saved data is found / file is not available\\n";
        return;
    }

    // Clear existing data
    while (airportHead) {
        Airport *atemp = airportHead;
        airportHead = atemp->next;
        while (atemp->flightHead) {
            Flight *ftemp = atemp->flightHead;
            atemp->flightHead = ftemp->next;
            delete ftemp;
        }
        delete atemp;
    }
}

```

```
airportHead = NULL;
```

```
string line, code, flightnum, dest, status;
```

```
int flightcount;
```

```
// Modified parsing logic to read file format generated by saveToFile
```

```
while (getline(infile, line)) {
```

```
    if (line.find("Airport Code:") != string::npos) {
```

```
        code = line.substr(line.find(":") + 2);
```

```
        getline(infile, line); // Total Flights: X
```

```
        flightcount = stoi(line.substr(line.find(":") + 2));
```

```
        // Skip separator lines and headers
```

```
        getline(infile, line); // separator
```

```
        getline(infile, line); // headers
```

```
        getline(infile, line); // separator
```

```
        addAirport(code);
```

```
    if (flightcount > 0) {
```

```
        for (int i = 0; i < flightcount; i++) {
```

```
            if (!getline(infile, line)) break;
```

```
            if (line == "No flights scheduled.") {
```

```
                break; // No flights here
```

```
            }
```

```
        // Parsing flight info lines - flightNo, destination, status by fixed widths
```

```

        string flightNo = line.substr(0,15);
        string destination = line.substr(15,25);
        string status = line.substr(40);

        // Trim trailing spaces
        flightNo.erase(flightNo.find_last_not_of(" \n\r\t")+1);
        destination.erase(destination.find_last_not_of(" \n\r\t")+1);
        status.erase(status.find_last_not_of(" \n\r\t")+1);

        addFlight_FromLoad(code, flightNo, destination, status);
    }
}
}

infile.close();
cout << "Data successfully loaded from " << filename << "\n";
}

Airport *findAirport(string code) {
    Airport *temp = airportHead;
    while (temp) {
        if (temp->airportCode == code)
            return temp;
        temp = temp->next;
    }
    return NULL;
}

~ATC() {

```

```

while (airportHead) {
    Airport *aTemp = airportHead;
    airportHead = airportHead->next;

    while (aTemp->flightHead) {
        Flight *fTemp = aTemp->flightHead;
        aTemp->flightHead = fTemp->next;
        delete fTemp;
    }
    delete aTemp;
}
};

```

// Main function

```

int main() {
    ATC control;
    int choice;
    string code, flightNum, dest;
    string filename = "atc_data.txt";

    do {
        cout << "\n--- Air Control Menu ---\n";
        cout << "1. Add Airport\n";
        cout << "2. Remove Airport\n";
        cout << "3. Arriving Flight\n";
        cout << "4. Departing Flight\n";
        cout << "5. Update Flight\n";
        cout << "6. Display All\n";
        cout << "7. Save to file\n";
    }
}

```

```
cout << "8. Load from file\n";  
cout << "0. Exit\n";  
cout << "Enter choice: ";  
cin >> choice;  
  
switch (choice) {  
    case 1:  
        cout << "Enter airport code: ";  
        cin >> code;  
        control.addAirport(code);  
        break;  
    case 2:  
        cout << "Enter airport code to remove: ";  
        cin >> code;  
        control.removeAirport(code);  
        break;  
    case 3:  
        cout << "Enter airport code: ";  
        cin >> code;  
        cout << "Enter flight number: ";  
        cin >> flightNum;  
        cout << "Enter destination: ";  
        cin >> dest;  
        control.addFlight(code, flightNum, dest);  
        break;  
    case 4:  
        cout << "Enter airport code: ";  
        cin >> code;  
        cout << "Enter flight number to delete: ";  
        cin >> flightNum;
```

```

        control.deleteFlight(code, flightNum);

        break;
case 5:
    cout << "Enter airport code: ";
    cin >> code;
    cout << "Enter flight number to update: ";
    cin >> flightNum;
    cout << "Enter new destination: ";
    cin >> dest;
    control.updateFlight(code, flightNum, dest);
    break;
case 6:
    control.display();
    break;
case 7:
    control.saveToFile(filename);
    break;
case 8:
    control.loadFromFile(filename);
    break;
case 0:
    cout << "Exiting...\n";
    break;
default:
    cout << "Invalid choice!\n";
}

} while (choice != 0);

return 0; }

```

Output:-

```
khushi-dudhalkar@khushi-dudhalkar-HP-Laptop-15-fd1xxx:~$ cd Desktop/DSA-FA1/Case\ Study/  
khushi-dudhalkar@khushi-dudhalkar-HP-Laptop-15-fd1xxx:~/Desktop/DSA-FA1/Case Study$ g++ Air_up2.cpp  
khushi-dudhalkar@khushi-dudhalkar-HP-Laptop-15-fd1xxx:~/Desktop/DSA-FA1/Case Study$ ./a.out
```

```
--- Air Control Menu ---  
1. Add Airport  
2. Remove Airport  
3. Arriving Flight  
4. Departing Flight  
5. Update Flight  
6. Display All  
7. Save to file  
8. Load from file  
0. Exit  
Enter choice: 1  
Enter airport code: DEL  
Airport DEL added.
```

```
--- Air Control Menu ---  
1. Add Airport  
2. Remove Airport  
3. Arriving Flight  
4. Departing Flight  
5. Update Flight  
6. Display All  
7. Save to file  
8. Load from file  
0. Exit  
Enter choice: 3  
Enter airport code: DEL  
Enter flight number: AI601  
Enter destination: Delhi  
Flight AI601 to Delhi added to DEL.
```

```
--- Air Control Menu ---  
1. Add Airport  
2. Remove Airport  
3. Arriving Flight  
4. Departing Flight  
5. Update Flight  
6. Display All  
7. Save to file  
8. Load from file  
0. Exit  
Enter choice: 3  
Enter airport code: DEL  
Enter flight number: SG8165  
Enter destination: Delhi  
Flight SG8165 to Delhi added to DEL.
```

```
--- Air Control Menu ---  
1. Add Airport  
2. Remove Airport  
3. Arriving Flight  
4. Departing Flight  
5. Update Flight  
6. Display All  
7. Save to file  
8. Load from file  
0. Exit  
Enter choice: 3  
Enter airport code: DEL  
Enter flight number: 6E203  
Enter destination: Delhi  
Flight 6E203 to Delhi added to DEL.
```

--- Air Control Menu ---

1. Add Airport
2. Remove Airport
3. Arriving Flight
4. Departing Flight
5. Update Flight
6. Display All
7. Save to file
8. Load from file
0. Exit

Enter choice: 1

Enter airport code: CCU

Airport CCU added.

--- Air Control Menu ---

1. Add Airport
2. Remove Airport
3. Arriving Flight
4. Departing Flight
5. Update Flight
6. Display All
7. Save to file
8. Load from file
0. Exit

Enter choice: 3

Enter airport code: CCU

Enter flight number: QP1395

Enter destination: Kolkata

Flight QP1395 to Kolkata added to CCU.

--- Air Control Menu ---

1. Add Airport
2. Remove Airport
3. Arriving Flight
4. Departing Flight
5. Update Flight
6. Display All
7. Save to file
8. Load from file
0. Exit

Enter choice: 3

Enter airport code: CCU

Enter flight number: 9I531

Enter destination: Kolkata

Flight 9I531 to Kolkata added to CCU.

--- Air Control Menu ---

1. Add Airport
2. Remove Airport
3. Arriving Flight
4. Departing Flight
5. Update Flight
6. Display All
7. Save to file
8. Load from file
0. Exit

Enter choice: 6

Airport: DEL | Total Flights: 3

Flight: AI601 -> Delhi | Status: On Time

Flight: SG8165 -> Delhi | Status: On Time

Flight: 6E203 -> Delhi | Status: On Time

Airport: CCU | Total Flights: 2

Flight: QP1395 -> Kolkata | Status: On Time

Flight: 9I531 -> Kolkata | Status: On Time

```
--- Air Control Menu ---
1. Add Airport
2. Remove Airport
3. Arriving Flight
4. Departing Flight
5. Update Flight
6. Display All
7. Save to file
8. Load from file
0. Exit
Enter choice: 4
Enter airport code: CCU
Enter flight number to delete: 9I531
Flight 9I531 deleted from CCU.
```

```
--- Air Control Menu ---
1. Add Airport
2. Remove Airport
3. Arriving Flight
4. Departing Flight
5. Update Flight
6. Display All
7. Save to file
8. Load from file
0. Exit
Enter choice: 5
Enter airport code: DEL
Enter flight number to update: AI601
Enter new destination: Pune
Flight AI601 updated.
```

```
--- Air Control Menu ---
1. Add Airport
2. Remove Airport
3. Arriving Flight
4. Departing Flight
5. Update Flight
6. Display All
7. Save to file
8. Load from file
0. Exit
Enter choice: 6

Airport: DEL | Total Flights: 3
Flight: AI601 -> Pune | Status: On Time
Flight: SG8165 -> Delhi | Status: On Time
Flight: 6E203 -> Delhi | Status: On Time

Airport: CCU | Total Flights: 1
Flight: QP1395 -> Kolkata | Status: On Time
```

```
--- Air Control Menu ---
1. Add Airport
2. Remove Airport
3. Arriving Flight
4. Departing Flight
5. Update Flight
6. Display All
7. Save to file
8. Load from file
0. Exit
Enter choice: 7
Data successfully saved to: atc_data.txt
```

```
atc_data.txt x
=====
Airport Code: DEL
Total Flights: 3
-----
Flight No      Destination      Status
-----
AI601          Pune              On Time
SG8165         Delhi             On Time
6E203          Delhi             On Time
=====

=====
Airport Code: CCU
Total Flights: 1
-----
Flight No      Destination      Status
-----
QP1395         Kolkata          On Time
=====
```

```
--- Air Control Menu ---
1. Add Airport
2. Remove Airport
3. Arriving Flight
4. Departing Flight
5. Update Flight
6. Display All
7. Save to file
8. Load from file
0. Exit
Enter choice: 8
Airport DEL added.
Airport CCU added.
Data successfully loaded from atc_data.txt
```

```
--- Air Control Menu ---
1. Add Airport
2. Remove Airport
3. Arriving Flight
4. Departing Flight
5. Update Flight
6. Display All
7. Save to file
8. Load from file
0. Exit
Enter choice: 0
Exiting...
```

Conclusion:-

This air traffic control system effectively demonstrates the use of **linked lists** and **object-oriented programming** to manage dynamic data for airports and flights. It integrates **memory management**, **data limits**, and **file persistence**, making it a functional simulation of real-world air traffic operations.